Hello, my name is Gulam Mustafa.

I am originally from Munger, Bihar. I completed my Bachelor of Computer Applications (BCA) from Echelon Institute of Technology, Faridabad.

Currently, I am working as a Reporting Analyst at EXL Service in Noida, and I have been working there for almost 2 years.

When I joined EXL, my main responsibility was creating reports using Microsoft Excel and automating daily tasks with VBA. I developed automated dashboards, reports, and macros that reduced manual work by around 70%. This also helped save more than 15 hours of work every week for the team.

One of my main projects is the Pharmacy Redaction Tool. It is developed using Microsoft Access and VBA. The purpose of this tool is to automatically identify and hide sensitive patient and healthcare information before the documents are shared. This helps the company follow healthcare data privacy rules and reduces manual effort. The tool also validates NDC drug codes, provides user-friendly forms for data entry, and automatically creates patient-specific redacted documents. It is a real application that is actively used by the business team.

Apart from this, I have also worked on a Generative AI proof-of-concept project called InsightsTalk AI. It is a healthcare claims analytics dashboard where users can ask questions in simple English. The application automatically generates SQL queries, retrieves data from Databricks, and displays the results in charts and dashboards. I built this project using Python, Databricks, LangChain, Groq API, and Streamlit. Although it is currently a proof of concept and not in production, it helped me gain practical experience in Generative AI, Retrieval-Augmented Generation (RAG), prompt engineering, and LLM-based applications.

Deep Dive — InsightsTalk AI (your headline project)

This is the project every interviewer will drill into. Be ready to explain it **architecturally**, not just as a bullet list.

What it does

A healthcare claims analytics dashboard where a business user can type a question in plain English (e.g., *"Show me claims with anomalies in the last 30 days for Region X"*) and get back an answer, a chart, and a root-cause explanation — without writing SQL.

Architecture (explain it as a pipeline)

1. **Data layer — Databricks SQL**
    - Claims data lives in a Databricks SQL warehouse.
    - Uses DESCRIBE TABLE calls to dynamically retrieve schema, so the LLM always knows current column names/types instead of relying on hardcoded schema.
2. **NL-to-SQL layer — LangChain + Groq API**
    - User's natural language question → passed through a LangChain-orchestrated prompt.
    - Groq API used for **fast LLM inference** (Groq is known for very low-latency inference via its LPU hardware — good talking point on why you chose it: speed for interactive dashboard use).
    - The LLM generates a SQL query against the retrieved schema.
3. **Structured output — Pydantic**
    - LLM responses are constrained to structured JSON using **Pydantic schema validation**, so outputs are type-safe and predictable before being used downstream (e.g., to populate a chart or trigger an alert) — this prevents "hallucinated" or malformed responses from breaking the app.
4. **Analytics layer**
    - Automated **anomaly detection** on claims (flagging outliers in claim volume/amount/patterns).
    - **Predictive risk flagging** using threshold-based rules combined with LLM-generated insight cards explaining *why* something was flagged (root cause analysis).
5. **Front end — Streamlit**

- Multi-tab layout, session state management (so filters/context persist across interactions), KPI "flip cards" with CSS animations, and a conversational chat interface for the NL-to-SQL feature.
- Real-time filter propagation — when a user changes a filter, it cascades across all visible KPIs/charts.

Good answers to likely questions

- **"Why RAG and not just a fine-tuned model?"** → RAG lets you query live, constantly-updating claims data without retraining a model every time the underlying data changes. It also grounds the LLM's SQL generation in the actual current schema, reducing hallucination.
- **"How do you prevent bad/unsafe SQL generation?"** → Talk about schema-grounding via DESCRIBE TABLE, Pydantic validation, and (if true) any query safety checks like read-only DB roles or query whitelisting. *(If you added guardrails like this, mention them — if not, it's a good thing to say you'd add.)*
- **"How did you pick Groq over OpenAI/Anthropic APIs?"** → Latency — Groq's inference hardware is optimized for speed, which matters for a responsive dashboard experience.
- **"What was the hardest technical challenge?"** → A strong answer here is usually: getting NL-to-SQL to reliably generate *correct* SQL against a real, sometimes messy healthcare schema, and handling ambiguous user questions.

4. Deep Dive — Pharmacy Redaction Tool

What it does

A compliance-focused desktop tool (Microsoft Access + VBA) that automates identifying and redacting sensitive pharmacy/drug data from patient records before documents are shared.

Key technical points to mention

- **NDC (National Drug Code) validation & matching logic** — cross-referencing incoming records against sensitive drug/group code datasets to flag which records need redaction. Good to explain NDC codes are standardized identifiers for drugs in the US, so matching logic likely involved exact/fuzzy matching against reference lists.
- **Forms-driven workflow** — built multiple user forms in Access for patient data upload, redaction management, and reporting — shows UI/UX thinking even in a legacy tool stack.
- **Automated PDF generation** — patient-specific documents generated automatically post-redaction, cutting manual processing time.
- **Compliance angle** — frame this as HIPAA-adjacent data privacy work (healthcare data protection), which is a strong selling point if applying to healthcare-adjacent roles.

Talking point: "This project shows I can work in legacy enterprise tooling (Access/VBA) just as comfortably as modern AI stacks — which matters in large orgs where not everything is greenfield."

---

5. Skills — Explained with Context (not just a list)

AI / LLM / RAG

- **LangChain** — used for prompt orchestration: chaining together schema retrieval → prompt construction → LLM call → structured parsing.
- **Groq API** — fast LLM inference provider; used for low-latency responses in an interactive app.
- **Pydantic** — Python data validation library; used to enforce structured JSON output from LLMs so downstream code can trust the shape of the data.
- **NL-to-SQL pipelines** — translating user questions into executable SQL grounded in real-time schema.
- **Prompt engineering** — designing prompts that reliably produce correct SQL/JSON, likely including few-shot examples, schema injection, and output-format constraints.

Python

- **Pandas** for data manipulation/analysis.
- Building AI pipelines that combine LangChain orchestration with Pydantic-validated outputs.
- Process automation scripts (likely replacing manual reporting tasks from Phase 1).

SQL & Databricks

- Complex query writing for extraction/transformation.
- Databricks SQL warehouse integration — connecting a Python/Streamlit app to a cloud data warehouse.
- Real-time SQL generation via LLM (the NL-to-SQL feature).
- Dynamic schema retrieval via DESCRIBE TABLE.

Streamlit

- Multi-tab dashboard layouts, session state (Streamlit's st.session_state for persisting values across reruns), custom CSS for KPI "flip card" animations, and chat-style interfaces (st.chat_message/st.chat_input pattern).

Excel (foundational strength)

- Advanced formulas (VLOOKUP/XLOOKUP, INDEX-MATCH), Pivot Tables, Power Query for ETL-style data prep, conditional formatting, and **VBA macros** for automating repetitive reporting tasks — the foundation that later fed into your automation-to-AI transition.

---

6. Anticipated Interview Questions + Suggested Angles

Question	Angle to take
"Walk me through InsightsTalk AI end to end"	Use the 5-step pipeline above (Data → NL-to-SQL → Structured Output → Analytics → UI)
"What's RAG and why does it matter here?"	Grounding LLM answers in real, current data (schema/claims) instead of relying purely on the model's training knowledge
"How do you ensure LLM outputs are reliable?"	Pydantic schema validation + structured JSON output + prompt engineering
"What would you improve about InsightsTalk AI?"	Good options: query safety guardrails, caching for repeated queries, evaluation/testing framework for SQL accuracy, user feedback loop
"How does your Excel/VBA background help your AI work?"	Deep understanding of the actual business reporting pain points you're now automating with AI — you're not building AI in a vacuum
"Tell me about a challenge in the Pharmacy Redaction Tool"	Handling sensitive data correctly (compliance-critical), NDC matching logic edge cases

---

7. Quick Reference — One-Liners for Each Skill (LinkedIn/resume bullet style)

- Built a **RAG-based NL-to-SQL healthcare analytics platform** (InsightsTalk AI) using LangChain, Groq API, Databricks SQL, and Streamlit.
- Reduced manual reporting effort by **70% (15+ hrs/week saved)** through Excel/VBA automation.

- Designed **Pydantic-validated structured LLM output pipelines** to ensure reliable, type-safe AI responses.
- Automated **sensitive healthcare data redaction** (NDC code validation) via an Access/VBA tool, improving compliance.
- Implemented **anomaly detection and threshold-based risk alerting** on claims data.